

Intro to C

- C
 - Evolved by Dennis Ritchie from two previous programming languages, BCPL and B
 - Used to develop UNIX
 - Used to write modern operating systems
 - Hardware independent (portable)
 - By late 1970's C had evolved to "Traditional C"
- Standardization
 - Many slight variations of C existed, and were incompatible
 - Committee formed to create a "unambiguous, machine-independent" definition
 - Standard created in 1989, updated in 1999

The C Standard Library

- C programs consist of pieces/modules called functions
 - A programmer can create his own functions
 - Advantage: the programmer knows exactly how it works
 - Disadvantage: time consuming
 - Programmers will often use the C library functions
 - Use these as building blocks
 - Avoid re-inventing the wheel
 - If a premade function exists, generally best to use it rather than write your own
 - Library functions carefully written, efficient, and portable

C and C++

- C++
 - Superset of C developed by Bjarne Stroustrup at Bell Labs
 - "Spruces up" C, and provides object-oriented capabilities
 - Object-oriented design very powerful
 - 10 to 100 fold increase in productivity
 - Dominant language in industry and academia

First Program

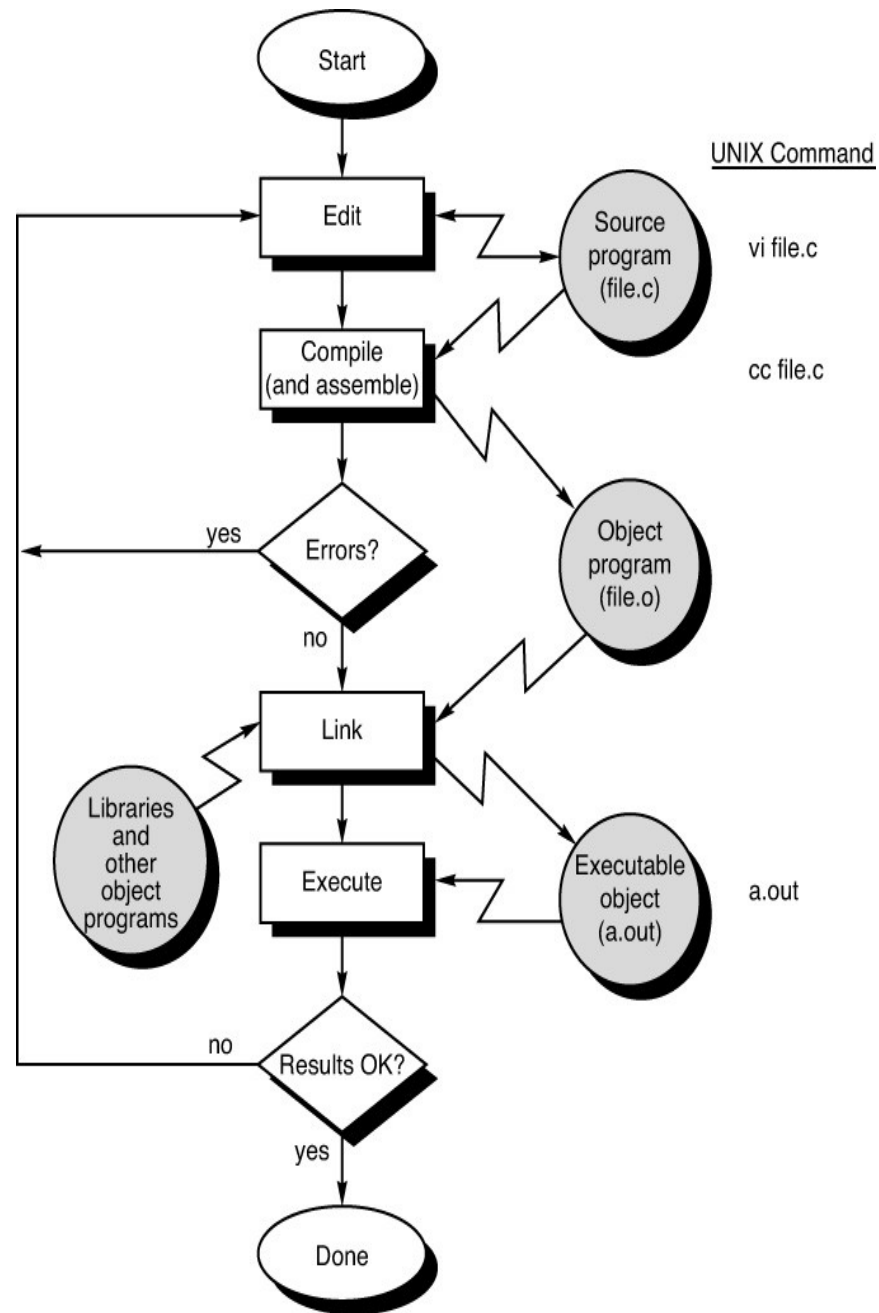
```
/* prog2_1.c
   this is the very first program in the book
   Program 2.1, on page 11
   which is explained in detail in Chapter 2
*/

#include <stdio.h>

// starting point of every program is the main function
int main (void) {

    printf("\nProgramming is fun!\n");

    return 0;    // tells OS that the program
                 // ran successfully
}
```



Incremental Development

stmt1
stmt2
stmt3
stmt4
stmt5
stmt6
stmt7
stmt8
stmt9
stmt10
stmt11
stmt12
stmt13
stmt14
stmt15
stmt16
stmt17
stmt18
stmt19
stmt20
stmt21
stmt22
stmt23
stmt24
stmt25

VS

stmt1
stmt2
stmt3 compile & run
stmt4
stmt5 compile & run
stmt6
stmt7
stmt8
stmt9
stmt10 compile & run
stmt11
stmt12
stmt13 compile & run
stmt14
stmt15 compile & run
stmt16
stmt17
stmt18
stmt19
stmt20
stmt21 compile & run
stmt22
stmt23
stmt24
stmt25 compile & run

First Program – Compiler Errors

```
/* prog2_1.c
   this is the very first program in the book
   Program 2.1, on page 11
   which is explained in detail in Chapter 2
*/

#include <stdio.h>

// starting point of every program is the main function
int main (void) {

    printf("\nProgramming is fun!\n")

    return 0;    // tells OS that the program
                 // ran successfully
}
```

```
origProg2_1.c: In function 'main':
origProg2_1.c:14:2: error: expected ';' before 'return'
    return 0;
    ^
```

Compiler Tries

stmt1

stmt2

stmt3 - - line with error

stmt4

stmt5 < - compiler message: Error at line 5

stmt6

- Programmer should examine line 5 *and earlier lines*

Short Program Loaded With Errors

```
#include <stdio.h>

int main() {
    int a = 2 b = 8, c = -3, result;

    result = 6 + b a * c;
    printf("result is: %d\n", result)

    result = a - b + c * 5);
    printf"result is: %d\n", result);

    result ((a + b) + 3 * c);
    printf(result is: %d\n", result);

    return 0:
}
```

Fixing Compiler Errors

```
predeclared.c:1: function 'main':
predeclared.c:6:12: error: expected ';' or ')' before 'W'
int a = 2 * b + 5, c = -3, result;
^

predeclared.c:6:2 error: 'result' undeclared (first use in this function)
result = 6 + b - 5; a * c;
^

predeclared.c:6:2 note: each undeclared identifier is reported only once for each function it appears in
predeclared.c:6:15: error: 'W' undeclared (first use in this function)
result = 6 + b - 5; a * c;
^

predeclared.c:6:15: error: expected ')' before 'W'
result = 6 + b - 5; a * c;
^

predeclared.c:7:8 warning: missing terminating ' character
printf("result is: %d\n", result);
^

predeclared.c:7:8 error: missing terminating " character
printf("result is: %d\n", result);
^

predeclared.c:9:15: error: 'c' undeclared (first use in this function)
result = a - b + c * 5;
^

predeclared.c:10:8: error: expected ')' before string constant
printf("result is: %d\n", result);
^

predeclared.c:10:22: error: expected statement before '}' token
printf("result is: %d\n", result);
^

predeclared.c:12:2 warning: implicit declaration of function 'main' [-Wimplicit-function-declaration]
result = (a + b) * 3 * c;
^

predeclared.c:12:16: error: expected ')' before 'W'
printf("result is: %d\n", result);
^

predeclared.c:13:16: error: stray '\n' in program
predeclared.c:13:24 warning: missing terminating " character
printf("result is: %d\n", result);
^

predeclared.c:13:16: error: missing terminating " character
printf("result is: %d\n", result);
^

predeclared.c:13:9: warning: passing argument 1 of 'printf' from incompatible pointer type [W incompatible-pointer-type]
printf("result is: %d\n", result);
^

In file included from predeclared.c:5:
./include/stddef.h:20:12: note: expected 'char*' declared but argument is of type 'int' [%W]
extern int printf (const char *__format, ...);
^

predeclared.c:13:9: warning: format not a string literal and no format arguments [-Wformat-security]
printf("result is: %d\n", result);
^

predeclared.c:16:1: error: expected ')' before '}' token
}
```

Short Program Loaded With Errors

```
#include <stdio.h>

int main() {
    int a = 2 b = 8, c = -3, result;

    result = 6 + b a * c;
    printf("result is: %d\n", result)

    result = a - b + c * 5);
    printf"result is: %d\n", result);

    result ((a + b) + 3 * c);
    printf(result is: %d\n", result);

    return 0:
}
```

Good Practice For Fixing Errors

do {

1. Focus on FIRST error message, ignoring the rest.
2. Find that error, either on that line specified or somewhere before it, and fix it.
3. Re-compile

} (repeat while more errors);

Compiler Warnings

- Compile this way:
`gcc-6 -Wall prog.c`
 - `gcc-6` will use the more recent version of gcc, closer to what Mimir uses
 - if you just use `gcc` without the `-6`, it will use version 5.4.0
 - `-Wall` forces all warnings to be displayed
 - `prog.c` is the name of the program
- You should always **FIX THE WARNINGS**

What Is “Compilation” ?

HIGH LEVEL LANGUAGE:

```
// sumOfSquares.c

#include <stdio.h>

int main(void) {
    int sum = 0, i = 0;

    while( i <= 100 ) {
        sum = sum + (i * i);
        i++;
    }

    printf("sum is %i\n", sum);

    return 0;
}
```

What Is “Compilation” ?

ASSEMBLY LANGUAGE (LOW-LEVEL):

```
addiu    $29, $29, -32
sw       $31, 20($29)
sw       $4, 32($29)
sw       $5, 36($29)
sw       $0, 24($29)
sw       $0, 28($29)
lw       $14, 28($29)
lw       $24, 24($29)
multu    $14, $14
addiu    $8, $14, 1
slti     $1, $8, 101
sw       $8, 28($29)
mflo     $15
addu     $25, $24, $15
bne      $1, $0, -9
sw       $25, 24($29)
lui      $4, 4096
lw       $5, 24($29)
jal      1048812
addiu    $4, $4, 1072
lw       $31, 20($29)
addiu    $29, $29, 32
jr       $31
move     $2, $0
```

What Is “Compilation” ?

MACHINE LANGUAGE (LOW-LEVEL):

```
001001111011110111111111111100000
101011111011111100000000000010100
101011111010010000000000000100000
101011111010010100000000000100100
10101111101000000000000000011000
10101111101000000000000000011100
10001111101011100000000000011100
10001111101110000000000000011000
00000001110011100000000000011001
00100101110010000000000000000001
00101001000000010000000001100101
10101111101010000000000000011100
00000000000000000111100000010010
00000011000011111100100000100001
0001010000100000111111111110111
10101111101110010000000000011000
00111100000001000001000000000000
10001111101001010000000000011000
000011000001000000000000011101100
00100100100001000000010000110000
10001111101111110000000000010100
001001111011110100000000000100000
0000001111100000000000000001000
00000000000000000001000000100001
```

Other Compiler Flags

`gcc-6 prog.c -o prog`

will change the executable named `a.out` to the file named `prog`

`gcc-6 -S prog.c`

will create `.s` file (assembly code)

`gcc-6 -c prog.c`

will create `.o` file (object code)

`gcc-6 -E prog.c > ppf`

will create file called `ppf` containing preprocessor results (will be a `.i` file)

`gcc-6 -save-temps prog.c`

will create `.i`, `.s`, & `.o` files